CS25C03 Essential Computing LAB Manual

Eoc (Anna University)

Scan to open on Studocu

Studocu is not sponsored or endorsed by any college or university
Downloaded by Arun Kumar (updatedraze@gmail.com)

OFFICE SOFTWARE FOR DOCUMENTATION AND PRESENTATION GIVE AIM OF EXPRIMENT

ExpNo:01

Date:

Aim:

To study and use office software for creating and formatting **documentation** and **presentations**, including text editing, object insertion, formatting, collaboration, and exporting.

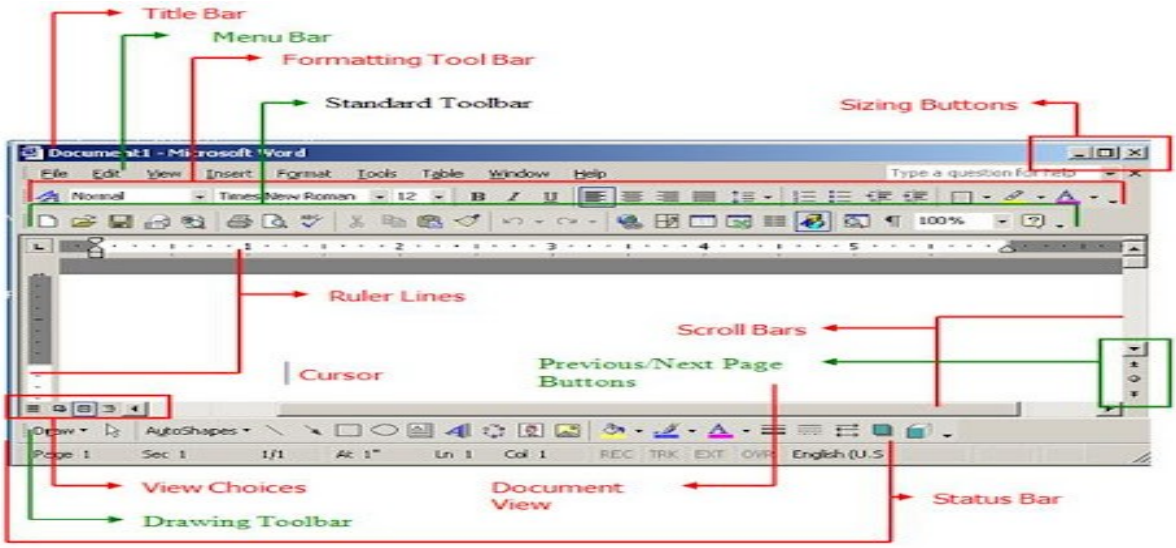
Software/Tools Required

- Microsoft Office (Word, PowerPoint) / LibreOffice (Writer, Impress) / Google Docs & Slides
- A computer with internet (for cloud-based tools)

Procedure

Part A: Documentation (Word Processor)

1. Open the word processor and create a **new document**.
2. Set **page layout** (size, margins, orientation).
3. Type **title** and **headings**, applying **styles** (Heading 1, Heading 2).
4. Insert **Header & Footer** with page number.
5. Format text with **bold, italics, alignment, bullets/numbering**.
6. Insert a **table** (3×5), format it, and insert **data**.
7. Insert an **image** with caption and alt text.
8. Insert a **chart/graph** using sample data.
9. Add **citations/references** and generate a **Table of Contents**.
10. Run **Spelling & Grammar check**.
11. **Save the document and export as PDF**.

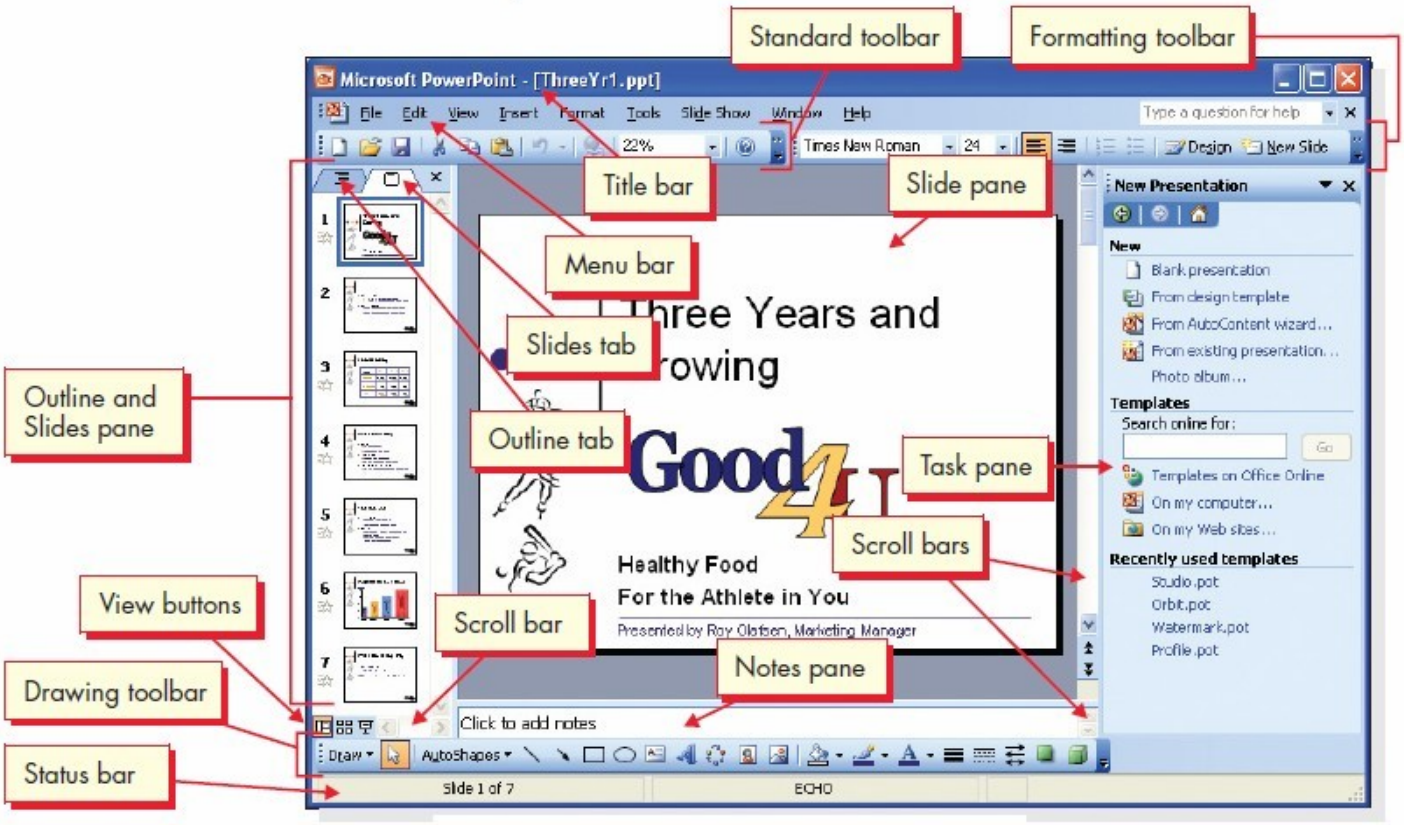


This document is available on studocu

Downloaded by Arun Kumar (updatedraze@gmail.com)

Part B: Presentation (Presentation Software)

1. Open presentation software and create a **new slideshow**.
2. Apply a **theme/template**.
3. Create slides:
    - Slide 1: Title slide (topic, name, date).
    - Slide 2: Objectives/Agenda.
    - Slides 3–6: Content with text, tables, images, charts.
    - Slide 7: Conclusion.
4. Use **layouts** for consistency.
5. Apply **transitions** and **simple animations**.
6. Add **speaker notes** to slides.
7. Run **spell check** and preview the slideshow.
8. Save as **.pptx/.odp** and export as **PDF**.



Downloaded by Arun Kumar (updatedraze@gmail.com)

This document is available on studocu

Downloaded by Arun Kumar (updatedraze@gmail.com)

Result:

The experiment to prepare and format documentation and presentation using office software was successfully carried out.

SPREADSHEETS FOR CALCULATION AND DATA VISUALIZATION

EXPNO:02

DATE:

Aim

To Use Spreadsheet Software (like Ms Excel, Libreoffice Calc, Or Google Sheets) For Performing Calculations And Creating Data Visualizations (charts/graphs) From Tabular Data.

Procedure

1. **Open the Spreadsheet Software** (MS Excel / Google Sheets / LibreOffice Calc).
2. **Enter the Data** in tabular form.
   - **Example:** Marks of students in different subjects.

Name	Math	Science	English
A	78	65	80
B	90	88	70
C	85	75	95
D	60	72	68

Downloaded by Arun Kumar (updatedraze@gmail.com)

3. **Apply Formulas for Calculation**

- Use =SUM(B2:D2) to calculate total marks.
- Use =AVERAGE(B2:D2) to calculate average marks.

4. Create Data Visualization (Charts)

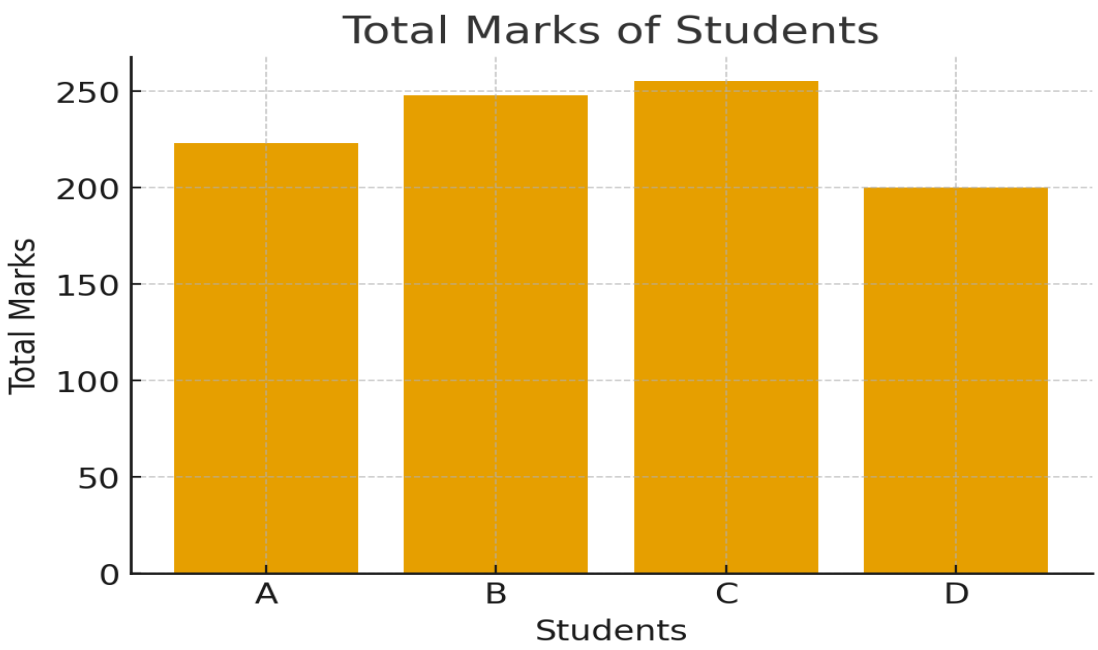
- Select the table.
- Go to **Insert → Chart**.
- **Choose a suitable chart type (Bar Chart / Pie Chart / Line Graph).**

5. Format the Chart with labels, legends, and titles.

6. Observe and Interpret the results.

Name	Math	Science	English	Total	Average
A	78	65	80	223	74.33
B	90	88	70	248	82.67
C	85	75	95	255	85.00
D	60	72	68	200	66.67

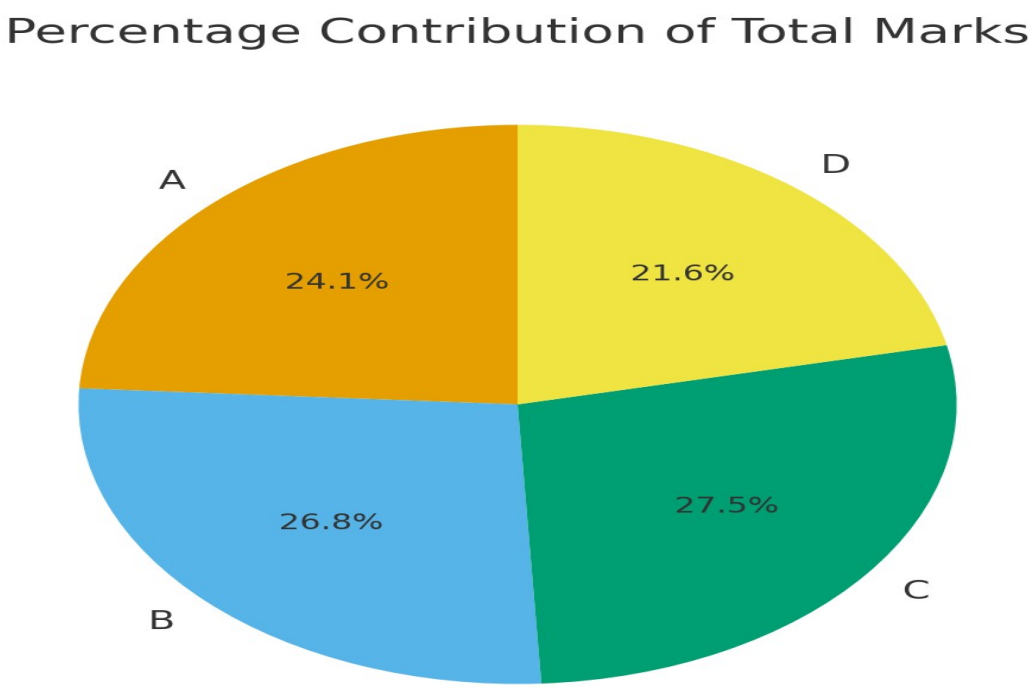
Graphical Visualization (Bar Chart Example):



This document is available on studocu

Downloaded by Arun Kumar (updatedraze@gmail.com)

Graphical Visualization (Pie Chart Example):



Downloaded by Arun Kumar (updatedraze@gmail.com)

Result:

The experiment was successfully carried out. Spreadsheet software was used to perform calculations (Total and Average) and create data visualizations (Bar Chart and Pie Chart). The results clearly represent student performance in both tabular and graphical formats.

SOLVING PROBLEMS BASED ON NUMBER SYSTEMS AND LOGIC

ExPNO:03

Date:

Aim

To use spreadsheet software for solving problems based on number systems (decimal, binary, octal, hexadecimal) and logical operations (AND, OR, NOT).

Procedure

1. Open Spreadsheet Software (MS Excel / LibreOffice Calc / Google Sheets).
2. Enter Data for conversion and logic operations.
    - Example (Number System Conversion):

Decimal	Binary	Octal	Hexadecimal
5			

This document is available on studocu

Downloaded by Arun Kumar (updatedraze@gmail.com)

10			
25			
50			

Example (Logic Operations):

A	B	A AND B	A OR B	NOT A
0	0			
0	1			
1	0			
1	1			

3. Apply Formulas:

- Binary: `=DEC2BIN(A2)`
- Octal: `=DEC2OCT(A2)`
- Hexadecimal: `=DEC2HEX(A2)`
- AND Logic: `=AND(A2,B2)`
- OR Logic: `=OR(A2,B2)`
- NOT Logic: `=NOT(A2)`

4. Insert Charts (Optional for visualization of number systems).

5. Observe and Record Results.

Sample Output (Table Simulation)

Number System Conversion Table

Decimal	Binary	Octal	Hexadecimal
5	101	5	5
10	1010	12	A
25	11001	31	19
50	110010	62	32

Logic Operation Table

Downloaded by Arun Kumar (updatedraze@gmail.com)

A	B	A AND B	A OR B	NOT A
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0
1	1	1	1	0

Result:

Problems based on number system conversions and logic operations were successfully solved using spreadsheet functions, and the results were verified.

Virtual Demonstration of Computational Thinking

EXPNO:4

Date:

Aim

To understand and demonstrate the core concepts of **Computational Thinking (CT)**decomposition, pattern recognition, abstraction, and algorithm design—through a virtual activity using a spreadsheet/software tool.

Procedure

1. **Open Spreadsheet / Online Tool** (Google Sheets, Excel, or block-based tools like Scratch/Blockly).
2. **Define a Problem** – Example: Find the sum of even numbers and visualize patterns.
3. **Apply Computational Thinking Steps**:
    - **Decomposition**: Break problem into smaller tasks (identify even numbers, add them).
    - **Pattern Recognition**: Notice that even numbers occur at every 2nd interval.

This document is available on studocu

Downloaded by Arun Kumar (updatedraze@gmail.com)

- **Abstraction**: Ignore unnecessary details, only keep number values.
- **Algorithm Design**: Create step-by-step solution using spreadsheet formulas.

4. **Implement in Spreadsheet**:
   - Enter numbers in column A.
   - Use =IF(MOD(A2,2)=0,"Even","Odd") to identify even/odd.
   - Use =SUMIF(B2:B10,"Even",A2:A10) to calculate sum of even numbers.
5. **Visualize Result**: Insert chart to show number patterns.

Sample Output Table

Number	Type
1	Odd
2	Even
3	Odd
4	Even
5	Odd

Formula Result:

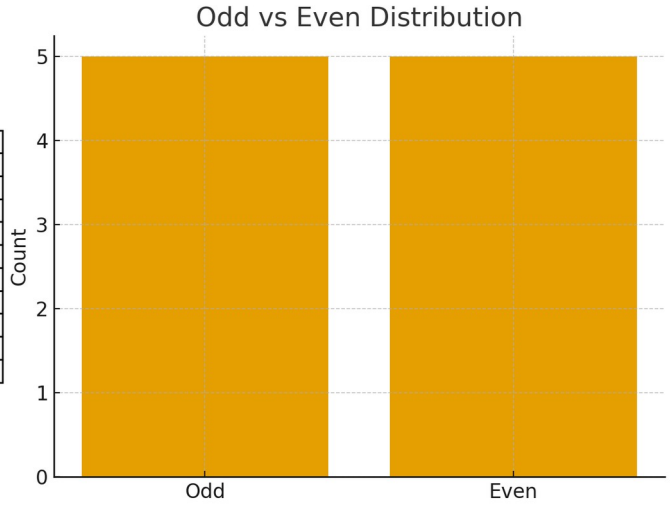
- Sum of Even Numbers = **12**

Downloaded by Arun Kumar (updatedraze@gmail.com)

Virtual Demonstration of Computational Thinking
Sum of Even Numbers = 30

Number Classification (Odd/Even)

Number	Type
1	Odd
2	Even
3	Odd
4	Even
5	Odd
6	Even
7	Odd
8	Even
9	Odd
10	Even



Here's the **output screenshot** for the virtual demonstration of computational thinking:

- Table showing numbers classified as **Odd/Even**
- Bar Chart showing **Odd vs Even distribution**
- Computation: **Sum of Even Numbers = 30**

This document is available on studocu

Downloaded by Arun Kumar (updatedraze@gmail.com)

Result

The problem was solved by applying **Computational Thinking concepts**:

- Decomposition (breaking problem into steps),
- Pattern Recognition (odd/even sequence),
- Abstraction (focusing on number type), and
- Algorithm Design (using formulas to compute).

Downloaded by Arun Kumar (updatedraze@gmail.com)

The sum of even numbers was successfully calculated and patterns were visualized.

FLOWCHART DESIGN FOR SIMPLE MATHEMATICAL PROBLEMS

EXPNO:5

Date:

Aim

To design and implement flowcharts for solving simple mathematical problems such as finding the largest number, checking even/odd, or calculating factorial.

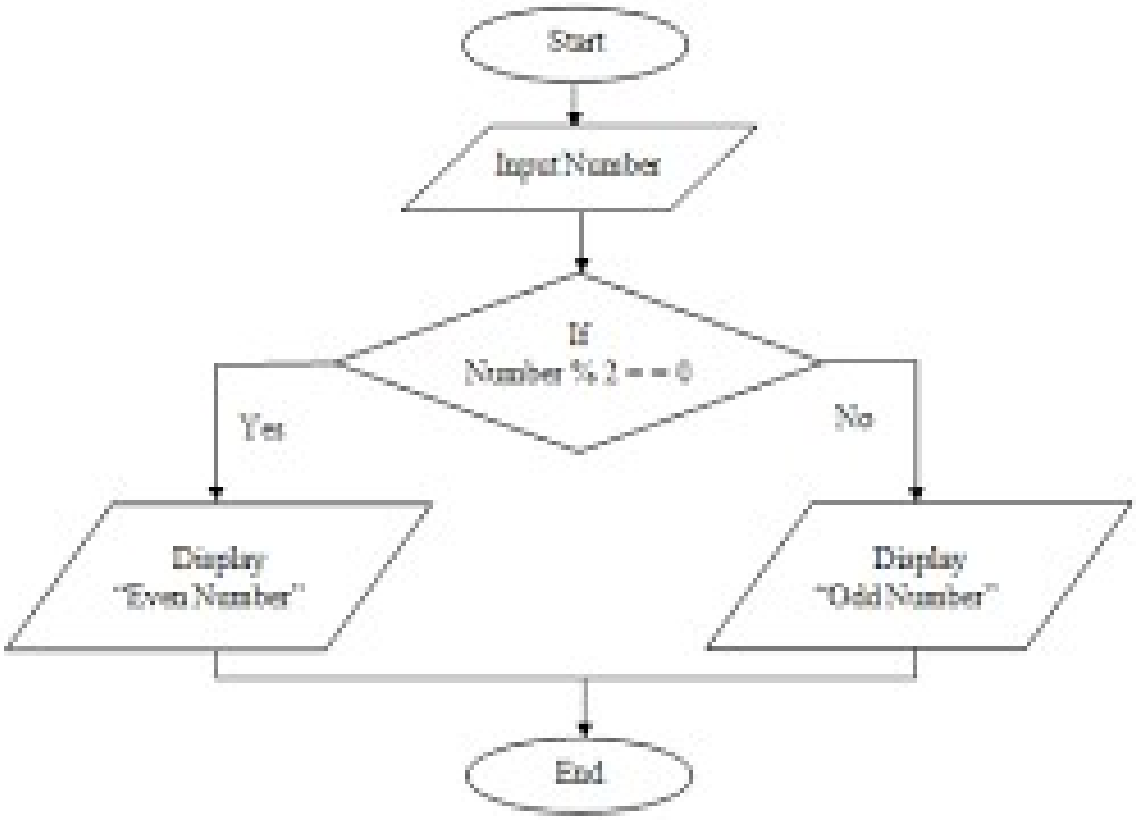
Procedure

1. **Select a Problem Statement**
   - Example 1: Check whether a number is even or odd.
   - Example 2: Find the factorial of a number.
2. **Identify Inputs and Outputs**
   - Input: Number n
   - Output: Result (Even/Odd or Factorial)
3. **Break Down Steps** (Algorithm)
   - For Even/Odd:
     - Start → Input number → Divide by 2 → If remainder = 0 → Even → Else Odd → End.
   - For Factorial:
     - Start → Input number → Initialize fact=1 → Repeat fact=fact*i for i=1 to n → Print fact → End.
4. **Draw the Flowchart** using standard symbols:
   - Oval: Start/End
   - Parallelogram: Input/Output
   - Rectangle: Process/Calculation
   - Diamond: Decision/Condition
5. **Test the Flowchart** with sample inputs.

This document is available on studocu

Downloaded by Arun Kumar (updatedraze@gmail.com)

Output (Flowchart Screenshot – Example: Even/Odd Check)



Downloaded by Arun Kumar (updatedraze@gmail.com)

Result

A flowchart was successfully designed for solving the **Even/Odd problem** using standard flowcharting symbols. The design clearly shows the sequence of steps, decision-making, and output.

CREATION OF FUNCTIONAL BLOCK FOR SIMPLE MATHEMATICAL PROBLEMS

EXPNO:

DATE:

Aim

To design and implement **functional blocks** (modular units) for solving simple mathematical problems such as **addition, subtraction, multiplication, division, and factorial calculation**.

Procedure

1. **Identify the Problem**
    - Example: Perform arithmetic operations on two numbers.
2. **Break Problem into Functional Blocks**
    - Block 1: Input numbers
    - Block 2: Addition
    - Block 3: Subtraction
    - Block 4: Multiplication
    - Block 5: Division
    - Block 6: Display Result
3. **Design Functional Blocks**
    - Each block performs only one task (modularity).
    - Input/Output flows between blocks.
4. **Represent the Blocks Visually** (block diagram).

This document is available on studocu

Downloaded by Arun Kumar (updatedraze@gmail.com)

5. **Test with Sample Input**
    - Example: a = 10, b = 5 → Results: Add=15, Sub=5, Mul=50, Div=2.

Output (Functional Block Diagram for Arithmetic Operations)

```
┌──────────────────┐
│  Input (a,b)  │
└──────────────────┘
          │
┌─────────▼─────────┐
│  Addition (a+b)│
└──────────────────┘
          │
┌─────────▼─────────┐
│ Subtraction (a-b) │
└──────────────────┘
          │
┌─────────▼──────────────┐
│ Multiplication (a*b) │
└────────────────────────┘
          │
┌─────────▼─────────┐
│ Division (a/b)  │
└──────────────────┘
```

Downloaded by Arun Kumar (updatedraze@gmail.com)

```
        │
┌───────▼──────────────────────┐
│  Display Results  │
└──────────────────────────────┘
```

Explanation

- **Input Block**: Takes two numbers as input (a, b).
- **Functional Blocks**:
  - Block 1 → Performs addition.
  - Block 2 → Performs subtraction.
  - Block 3 → Performs multiplication.
  - Block 4 → Performs division.

- **Output Block**: Displays all results.

Sample Run

Input:

a = 10, b = 5

Output:

Addition = 15

Subtraction = 5

Multiplication = 50

Division = 2

This document is available on studocu

Downloaded by Arun Kumar (updatedraze@gmail.com)

Result:

A functional block diagram was successfully created for solving basic arithmetic problems. The results were computed and displayed correctly.

DRAWING AND PAINTING OPERATIONS

EXPNO:07

DATE:

Aim

To demonstrate **drawing and painting operations** such as creating shapes (lines, rectangles, circles), filling them with colors, and combining them to form simple designs using computer graphics software.

Procedure

1. **Open Drawing Tool** (MS Paint, Tux Paint, LibreOffice Draw, or any graphics library).
2. **Drawing Operations**:
   - Draw basic shapes (Line, Rectangle, Circle, Polygon).
   - Combine multiple shapes to form a design (like a house, flower, or pattern).
3. **Painting Operations**:
   - Use Fill Tool (Bucket) to apply colors inside shapes.
   - Apply different brush styles or gradients.
4. **Save the Drawing** as an image file.
5. **Observe the Output:**A colorful illustration showing drawing + painting operations.

Step-by-Step Guide: Drawing and Painting Operations

1. Open the Tool

- Open **MS Paint** (Windows) or **LibreOffice Draw** (Linux).

Downloaded by Arun Kumar (updatedraze@gmail.com)

2. Draw the Base (House Walls)

- Select the **Rectangle Tool**.
- Draw a rectangle in the center (this will be the house walls).
- Use the **Fill Tool** (bucket icon) to paint it with a light color (e.g., wheat).

3. Draw the Roof

- Select the **Triangle / Polygon Tool**.
- Draw a triangle on top of the rectangle (for the roof).
- Fill the roof with **brown color**.

4. Draw the Door

- Draw a smaller rectangle at the bottom-center of the house.
- Fill it with **dark brown color**.

5. Draw Windows

- Draw **two small rectangles** (left and right side of the house).
- Fill them with **sky blue color** (to represent glass).

6. Draw the Sun

- Select the **Circle/Ellipse Tool**.
- Draw a circle in the top-right corner.
- Fill it with **yellow color**.

7. Draw the Ground

- Draw a long rectangle at the bottom of the canvas.
- Fill it with **green color** (to represent grass/ground).

8. Save the Drawing

- Go to **File → Save As → PNG/JPEG**.
- Save with the name drawing painting.png.

Flowchart: Drawing and Painting Operations

```
┌──────────────────────┐
│      Start           │
└──────────────────────┘
            │
            ▼
┌──────────────────────┐
│ Open MS Paint /      │
│ LibreOfficeDraw      │
└──────────────────────┘
```

This document is available on studocu

Downloaded by Arun Kumar (updatedraze@gmail.com)

```
┌──────────▼──────────┐
│ Draw Rectangle      │
│ (House Base)        │
└──────────┬──────────┘
           │
┌──────────▼──────────┐
│ Draw Triangle       │
│ (Roof)              │
└──────────┬──────────┘
           │
┌──────────▼──────────┐
│ Draw Door + Windows │
└──────────┬──────────┘
           │
┌──────────▼──────────┐
│ Draw Circle (Sun)   │
└──────────┬──────────┘
           │
┌──────────▼──────────┐
│ Draw Ground (Green) │
└──────────┬──────────┘
           │
┌──────────▼──────────┐
│ Apply Colors (Paint)│
└──────────┬──────────┘
           │
┌──────────▼──────────┐
│   Save Drawing      │
└──────────┬──────────┘
           │
┌──────────▼──────────┐
│       End           │
└─────────────────────┘
```

Downloaded by Arun Kumar (updatedraze@gmail.com)

Result:

The drawing and painting operations were successfully demonstrated by creating a simple colorful scenery using basic shapes and paint tools.

SCRATCH ANIMATION FOR CONDITIONAL AND LOOPING STATEMENTS

EXPNO:08

DATE:

Aim

To create a Scratch animation demonstrating the use of **conditional statements (if, if-else)** and **looping statements (repeat, forever)** for decision-making and repetitive actions.

Procedure

Open Scratch (online at scratch.mit.edu

Choose a sprite (e.g., Cat).

Set background (choose any stage background, e.g., “Blue Sky”).

Add looping behavior:

- Use a **forever loop** to make the sprite move continuously.
- Inside the loop, add **move 10 steps** and **if on edge, bounce**.

Add conditional statement:

- Use an **if-else block** to check a condition.
- Example:
  - If the sprite touches the edge, say **“I hit the wall!”**,
  - Else, say **“I am moving…”**.

Add another loop (repeat) for demonstration:

This document is available on studocu

Downloaded by Arun Kumar (updatedraze@gmail.com)

- Create an animation (e.g., sprite spins 10 times).
- Use repeat 10 → turn 15 degrees.

Run the program by clicking the green flag.

Sample Script (Pseudocode in Scratch blocks)

```
when green flag clicked
forever
move 10 steps
if on edge, bounce
 if touching edge
  say "I hit the wall!" for 2 secs
 else
  say "I am moving..." for 2 secs
end
```

• **Repeating Animations:**
Use loops to create repeating animations, such as continuously making a cat dance or fish swim.

• **Movement Patterns:**
Implement complex movement patterns, like spirals, zigzags, or figure-eights, using loops.

• **Musical Compositions:**
Compose music by repeating musical notes or melodies, creating catchy tunes or rhythms.

• **Interactive Games:**
Develop interactive games with loops to handle game mechanics like enemy movements, character actions, or continuous scoring.

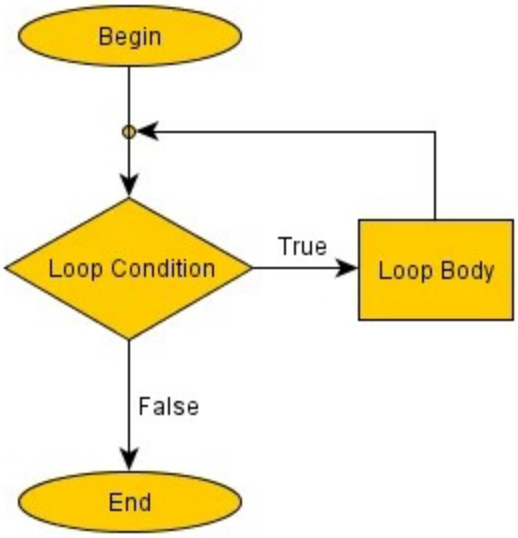
• **Countdowns and Timers:**
Create countdowns or timers using loops to display the time remaining in a game or for other time-dependent activities.

• **Educational Quizzes:**
Design educational quizzes or interactive lessons with loops to provide feedback and keep track of scores.

• **Randomisation:**
Generate random events or outcomes in your project, like random enemy spawns or random questions in a quiz.

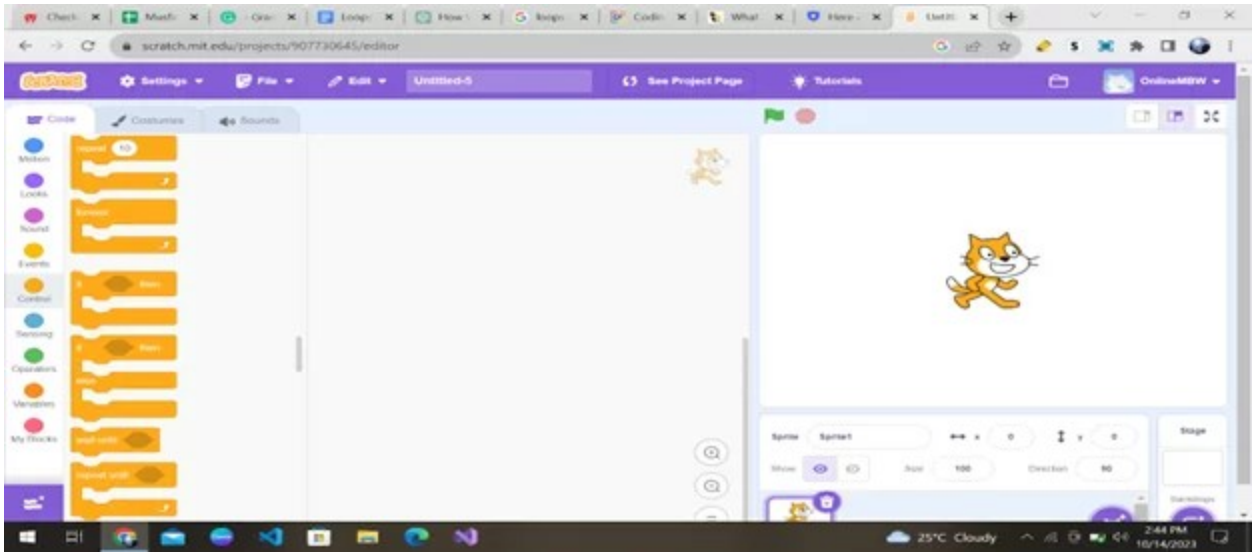
• **Data Processing:**
Use loops to process and manipulate data, like sorting items in a list or filtering data based on specific criteria.

Downloaded by Arun Kumar (updatedraze@gmail.com)



Types of Loops in Scratch

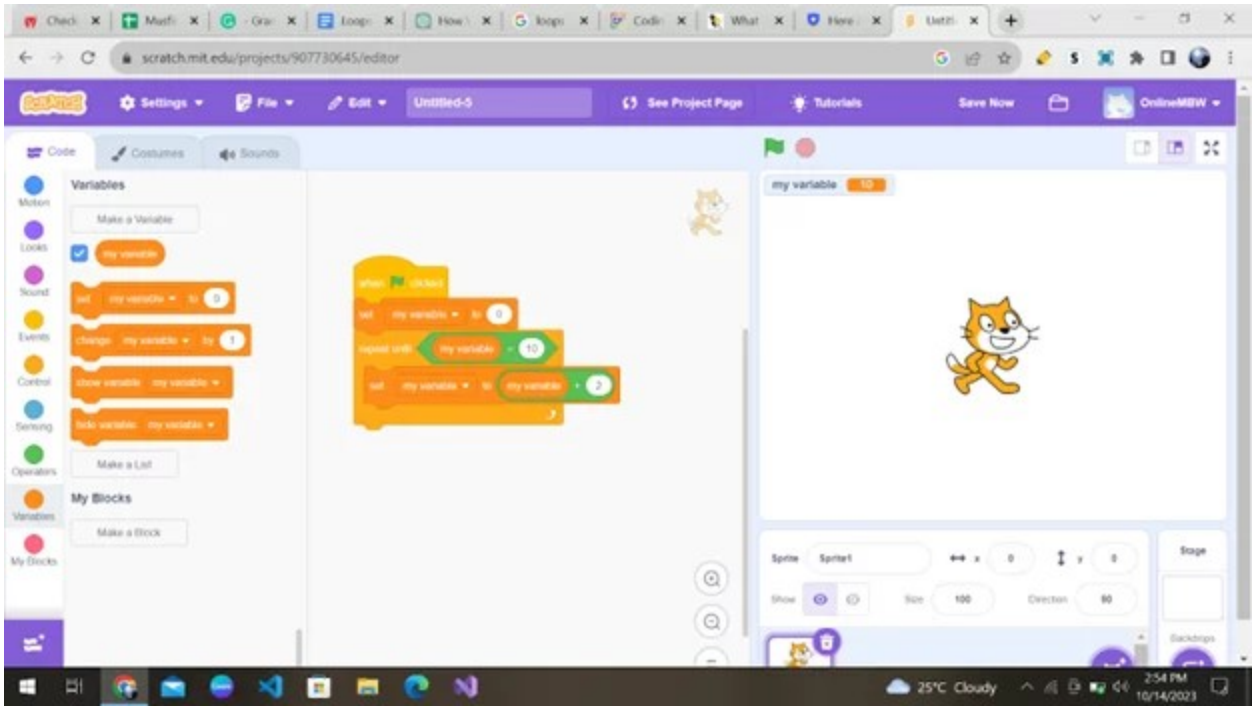
In the ‘Control’ section of block code, you can see 3 types of loops: repeat until, repeat x number of times, and forever loop. Each type of loop serves a different purpose, and it is important to know what each can do



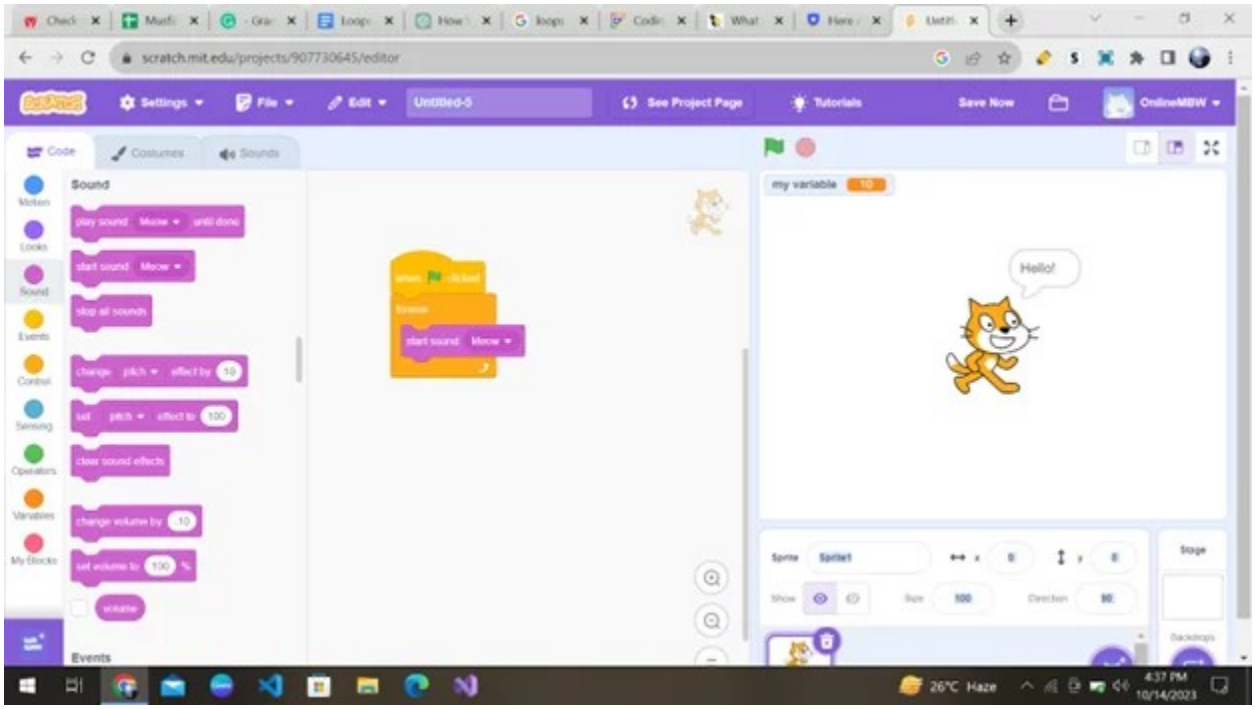
When the provided boolean statement is true, the blocks in this block loop until the code below them (if any) is executed.

This document is available on studocu

Downloaded by Arun Kumar (updatedraze@gmail.com)



This loop has no end and is useful for repeating an action or a set of actions forever. Like a never-ending story, this loop will repeat your code forever. You can use it for tasks like keeping a game running or making your cat dance endlessly



Downloaded by Arun Kumar (updatedraze@gmail.com)

RESULT:

Thus The Scratch Animation For Conditional And Looping Statements executed successfully.

This document is available on studocu

Downloaded by Arun Kumar (updatedraze@gmail.com)